CSA1413

COMPILER DESIGN

Name: Haresh Kumar N L

Reg No: 192425009

Lexical Analyzer Program

Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 1000
#define MAX_TOKEN 100

int isKeyword(const char *word)
{
   const char *keywords[] = {
      "auto", "break", "case", "char", "const",
      "continue", "default", "do", "double", "else",
      "enum", "extern", "float", "for", "goto",
      "if", "int", "long", "register", "return",
      "short", "signed", "sizeof", "static", "struct",
      "switch", "typedef", "union", "unsigned", "void",
      "volatile", "while", "inline", "restrict", "_Bool",
      "_Complex", "_Imaginary"
   };

   int count = sizeof(keywords) / sizeof(keywords[0]);
```

```
    for (int i = 0; i < count; i++)
    {
        if (strcmp(word, keywords[i]) == 0)
            return 1;
    }

    return 0;
}

int main(void)
{
    char input[MAX_INPUT];
    int i = 0;

    printf("Enter a C statement:\n");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    while (input[i] != '\0')
    {
        if (isspace((unsigned char)input[i]))
        {
            i++;
            continue;
        }

        /* Single-line and block comments */
        if (input[i] == '/' && input[i + 1] == '/')
```

```
    break;

if (input[i] == '/' && input[i + 1] == '*')
{
    i += 2;

    while (input[i] != '\0' &&
           !(input[i] == '*' && input[i + 1] == '/'))
        i++;

    if (input[i] == '*')
        i += 2;

    continue;
}

/* Identifier or keyword */
if (isalpha((unsigned char)input[i]) ||
    input[i] == '_')
{
    char word[MAX_TOKEN];
    int j = 0;

    while (isalnum((unsigned char)input[i]) ||
           input[i] == '_')
    {
        if (j < MAX_TOKEN - 1)
            word[j++] = input[i];
```

```
        i++;
    }

    word[j] = '\0';

    printf("%-15s %s\n", word,
           isKeyword(word) ? "Keyword" : "Identifier");
    continue;
}

/* Numeric constants */
if (isdigit((unsigned char)input[i]))
{
    char number[MAX_TOKEN];
    int j = 0;

    while (isalnum((unsigned char)input[i]) ||
           input[i] == '.' || input[i] == '_')
    {
        if (j < MAX_TOKEN - 1)
            number[j++] = input[i];

        i++;
    }

    number[j] = '\0';
    printf("%-15s Constant\n", number);
    continue;
}
```

```
/* String and character literals */
if (input[i] == '"' || input[i] == '\'')
{
   char quote = input[i];
   char literal[MAX_TOKEN];
   int j = 0;

   literal[j++] = input[i++];

   while (input[i] != '\0' && input[i] != quote)
   {
      if (input[i] == '\\' && input[i + 1] != '\0')
      {
         if (j < MAX_TOKEN - 1)
            literal[j++] = input[i];
         i++;
      }

      if (j < MAX_TOKEN - 1)
         literal[j++] = input[i];

      i++;
   }

   if (input[i] == quote)
   {
      if (j < MAX_TOKEN - 1)
         literal[j++] = input[i];
```

```
        i++;
    }

    literal[j] = '\0';
    printf("%-15s %s\n", literal,
           quote == '"' ? "String Literal" :
                          "Character Constant");
    continue;
}

/* Operators: longest match first */
{
    const char *operators[] = {
        ">>=", "<<=", "++", "--", "+=", "-=",
        "*=", "/=", "%=", "==", "!=", "<=",
        ">=", "&&", "||", "<<", ">>", "&=",
        "|=", "^=", "->", "+", "-", "*", "/",
        "%", "=", "<", ">", "!", "&", "|",
        "^", "~", "?"
    };

    int count = sizeof(operators) / sizeof(operators[0]);
    int found = 0;

    for (int k = 0; k < count; k++)
    {
        int len = (int)strlen(operators[k]);
```

```
            if (strncmp(input + i, operators[k], len) == 0)
            {
                printf("%-15.*s %s\n", len, input + i,
                       strcmp(operators[k], "=") == 0
                           ? "Assignment Operator"
                           : "Operator");

                i += len;
                found = 1;
                break;
            }
        }

        if (found)
            continue;
    }

    /* Special symbols */
    if (strchr("(){}[];,.:", input[i]) != NULL)
    {
        printf("%-15c Special Symbol\n", input[i]);
        i++;
        continue;
    }

    printf("%-15c Unknown Symbol\n", input[i]);
    i++;
}
```

```
    return 0;
}
```

Output:

